

Accord — Product Requirements Document (PRD)

Status

Draft v0.1

Purpose

This PRD defines the first product shape for Accord.

The goal is to convert the project thesis and research into a concrete implementation target. This document answers:

- who the product is for,
- what problem it solves first,
- what capabilities belong in v1,
- what does not belong in v1,
- how success should be measured,
- and what tradeoffs we are deliberately making.

This is a **v1 document**, not a full future roadmap.

1. Product Summary

Accord is an open protocol and developer SDK for **scoped context disclosure** and **human approval of high-impact AI actions**.

It helps developers building AI agents, copilots, and personal workflow tools add a consistent control layer that:

- declares what user context was used,
- scopes that context access,
- pauses risky actions for human approval,
- records outcomes,
- and makes those decisions inspectable and revocable.

Accord is infrastructure for **approval-aware personalization**.

2. Problem Statement

Developers building AI applications increasingly face the same trust problem:

- their applications want to use personal or persistent context,
- their applications want to take real actions,
- but there is no consistent, reusable way to disclose context usage and require approval before execution.

As a result, teams often build ad hoc solutions for:

- consent prompts,
- approval dialogs,
- policy rules,
- action logs,
- and revocation behavior.

These solutions are inconsistent, difficult to port across products, and often too shallow to serve as reliable infrastructure.

Accord exists to provide a **standardized, developer-friendly control layer** for this problem.

3. Product Vision

The long-term vision is:

Any AI application that uses personal context and takes meaningful action should be able to disclose what it used, ask for approval when required, and leave behind an auditable record.

In the long run, Accord could become a portable standard that multiple products and ecosystems build on.

But the first version should stay narrow:

Make it easy for developers to add structured context disclosure and action approval to AI apps without inventing the control plane from scratch.

4. Primary Goal for v1

v1 Goal

Enable a developer to integrate Accord into a local AI application so that:

1. the application can request scoped access to user context,
2. the application can disclose which context it used for a proposed action,

3. risky actions can be held until explicit approval,
4. the system can record approval outcomes,
5. and the user can inspect the resulting history in a minimal console.

That is the v1 win condition.

5. Target Users

Primary User

Developers building AI agents, copilots, and workflow tools who need:

- a reusable approval layer,
- structured disclosure of context usage,
- a policy hook for action gating,
- and auditability without building all of it themselves.

These users are likely to be:

- AI power users,
- agent-framework builders,
- indie developers,
- open-source contributors,
- or teams prototyping trustworthy AI behavior.

Secondary User

Technically inclined end users interacting with an Accord-enabled application.

They care about:

- knowing what context the application used,
- approving or rejecting actions,
- and being able to inspect or revoke those decisions.

Non-Target User for v1

- mainstream non-technical consumers,
- enterprise administrators,
- compliance teams,
- or large organizations needing hosted governance and identity integration.

Those are possible later audiences, but they are not the v1 focus.

6. Core User Stories

Developer Stories

1. As a developer, I want to define a context access request with clear scope so my AI app does not silently use user data.
2. As a developer, I want to attach a context manifest to an action proposal so the user can see what information influenced the action.
3. As a developer, I want a simple way to mark some actions as approval-required.
4. As a developer, I want an approval receipt I can check before executing a write or side effect.
5. As a developer, I want an audit trail so I do not need to design logging and approval storage myself.
6. As a developer, I want the system to run locally in development without requiring a hosted service.

End User Stories

1. As a user, I want to see what context the AI used before it takes a meaningful action.
 2. As a user, I want to approve, reject, or modify an action before it executes.
 3. As a user, I want to inspect prior approvals and denials later.
 4. As a user, I want to revoke context access or let it expire.
 5. As a user, I want the system to feel understandable, not magical.
-

7. v1 Use Cases

Use Case 1: Local Coding Agent

A developer uses a coding agent that reads project preferences and proposes file edits or Git actions.

Accord should support: - scope request for project preferences, - disclosure of what preferences were used, - approval before file write / commit / push, - and an audit record of the decision.

Use Case 2: Research-to-Publish Agent

A local research or writing agent uses stored writing preferences and publishing rules to draft content and propose publication actions.

Accord should support: - scope request for writing preferences, - context manifest for style or publishing rules used, - approval before publish / PR / export, - and a record of the approval result.

Use Case 3: Generic Local Workflow Action

A reference sample app proposes an external side effect such as saving, exporting, or updating a resource.

Accord should support the same runtime flow even when the specific agent logic differs.

8. Product Principles

1. Runtime trust over abstract theory

Every feature in v1 should reinforce the runtime moment where context and action meet.

2. Local-first by default

The first version should be easy to run, inspect, and understand on a local machine.

3. Explicit disclosure

Users should not have to guess what context influenced an action.

4. Approval as a first-class primitive

Approval is part of the normal execution flow, not an afterthought.

5. Minimal integration surface

The SDK should be easy to adopt without forcing developers into a large framework.

6. Protocol before polish

The protocol objects and runtime flow matter more than a flashy UI.

9. Functional Requirements for v1

FR1 — Scope Requests

The system must allow an AI application to request access to user context in a structured way.

A scope request should include at minimum: - requester identity, - context categories requested, - intended purpose, - requested duration or expiry, - and optional sensitivity indicators.

FR2 — Context Manifest

The system must allow an application to disclose which context was actually used in generating an action proposal.

A context manifest should include at minimum: - references to the context categories used, - provenance or source information where appropriate, - timestamps, - and the relation between context and the proposed action.

FR3 — Action Proposal

The system must support structured action proposals for operations that may require approval.

An action proposal should include at minimum: - action type, - target, - human-readable summary, - risk classification or approval hint, - associated context manifest, - and execution metadata.

FR4 — Approval Workflow

The system must support an approval decision flow where a user can: - approve, - reject, - or modify/annotate the proposal if that capability is exposed.

FR5 — Approval Receipt

The system must generate a machine-checkable approval outcome that can be referenced before execution.

FR6 — Policy Engine

The system must support policy evaluation that determines whether an action: - can proceed automatically, - must be held for approval, - or must be denied.

FR7 — Audit Logging

The system must record scope requests, action proposals, approval outcomes, and revocations in an inspectable history.

FR8 — Revocation / Expiry

The system must support revoking granted scope and expiring time-bounded permissions.

FR9 — Local Console

The system must provide a minimal local console where a user can: - inspect active scopes, - review action proposals, - approve or deny actions, - inspect history, - and revoke scopes.

FR10 — Developer SDK

The system must provide a TypeScript SDK with ergonomics good enough for local prototype integrations.

10. Non-Functional Requirements for v1

NFR1 — Local-First Operation

The core v1 system must run locally without requiring cloud infrastructure.

NFR2 — Inspectability

All protocol objects and runtime decisions should be viewable and understandable in development.

NFR3 — Simplicity

A developer should be able to perform a meaningful first integration with low setup overhead.

NFR4 — Extensibility

The spec and SDK should be structured to allow adapters and future expansion without breaking the core model.

NFR5 — Deterministic Enforcement Boundary

An action should not execute through Accord-controlled flow without satisfying the required approval or policy conditions.

NFR6 — Clear Failure Modes

If approval is missing, expired, or denied, the action must fail closed.

11. v1 Non-Goals

The following are explicitly out of scope for v1:

- building a new memory engine,
- ranking or retrieval optimization for stored context,
- Gmail / calendar / commerce integrations,
- a mobile app,
- hosted multi-tenant infrastructure,
- organization-wide governance,
- identity federation / SSO,
- enterprise compliance workflows,
- autonomous decision-making without a human boundary,
- or a complete end-user assistant product.

These are tempting expansions, but they would weaken the first version.

12. Key Objects in v1

The exact schema will be defined in the protocol spec, but v1 should revolve around these core objects:

- **Scope Request**

- **Granted Scope**
- **Context Manifest**
- **Action Proposal**
- **Approval Decision / Receipt**
- **Revocation Event**
- **Audit Record**

These objects form the core runtime grammar of the system.

13. Proposed v1 Components

Component A — Protocol Spec

Defines the object model and runtime semantics.

Component B — TypeScript SDK

Gives developers integration primitives.

Component C — Local Accord Server

Receives requests, evaluates policies, stores records, and exposes local APIs.

Component D — Policy Engine

Applies rules to determine whether an action is allowed, blocked, or approval-required.

Component E — Approval Console

Provides a local UI for review, approval, history, and revocation.

Component F — Sample Integrations

Shows how the system works in real flows.

14. Developer Experience Requirements

A developer integrating Accord should be able to:

1. install the SDK,
2. start a local server,
3. declare a scope request,
4. create an action proposal,
5. wait for approval or policy result,

6. check the approval receipt,
7. and only then execute the side effect.

The integration should feel like adding a control plane, not adopting an entire opinionated framework.

15. Success Metrics for v1

Because this is an early open-source infrastructure project, success should be measured primarily by **clarity and usability**, not scale.

Product Success Signals

- The protocol objects feel coherent and easy to explain.
- A developer can complete a sample integration in a few hours.
- The approval flow is understandable without reading deep internal logic.
- The console makes it obvious what context was used for a given proposal.
- The failure boundary is clear when approval is missing or denied.

OSS Success Signals

- Early contributors can understand the project structure quickly.
 - The README and sample apps make the purpose of the project obvious.
 - Developers see Accord as a reusable primitive, not just a demo.
-

16. Failure Conditions

Accord v1 would be failing if:

- it feels like a generic wrapper with no distinct runtime value,
 - it becomes mostly a UI demo instead of a protocol and SDK,
 - the approval flow is vague or optional in practice,
 - the scope model is too abstract to be useful,
 - or the project expands into too many adjacent concerns before the core model is solid.
-

17. Tradeoffs We Are Deliberately Making

Tradeoff 1: Narrow scope over feature breadth

We are choosing a sharp control-layer wedge instead of trying to solve all personalization or governance problems.

Tradeoff 2: Local-first over hosted convenience

We are prioritizing inspectability and rigor over immediate cloud polish.

Tradeoff 3: Developer-first over consumer-first

The first adopters should be builders, not mainstream end users.

Tradeoff 4: Approval clarity over autonomy hype

We are intentionally slowing down action execution where trust matters.

Tradeoff 5: Protocol rigor over flashy demos

The lasting value comes from the runtime model, not from demo theatrics.

18. Open Questions for the Next Document

These questions should be resolved in the protocol spec and architecture doc:

1. What exact fields are required in each protocol object?
 2. What is the minimal risk model for deciding approval-required vs auto-allow?
 3. How should policy rules be configured locally?
 4. What should the local persistence model look like?
 5. How should approval receipts be represented and verified?
 6. What is the smallest viable API surface for the SDK?
-

19. Final v1 Framing

Accord v1 is successful if it proves one thing clearly:

An AI application can use personal context and propose meaningful actions without becoming opaque or unchecked.

If we can make that real in a clean, local, open-source developer experience, we will have built something worth expanding.